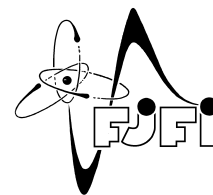




CZECH TECHNICAL UNIVERSITY IN PRAGUE
Faculty of Nuclear Sciences and Physical Engineering



Procedural generation of dungeons in Godot Engine

Procedurální generování dungeonů v Godot Engine

Bachelor's Degree Project

Author: **Robin Havlina**
Supervisor: **Ing. Petr Pauš, Ph.D.**
Language advisor: **Bc. Nathaniel Tobias Patton**
Academic year: 2025/2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Havlina** Jméno: **Robin** Osobní číslo: **519842**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávající katedra/ústav: **Katedra matematiky**
Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Procedurální generování dungeonů v Godot Engine

Název bakalářské práce anglicky:

Procedural generation of dungeons in Godot Engine

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Petr Pauš, Ph.D. katedra softwarového inženýrství FIT

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **31.10.2025**

Termín odevzdání bakalářské práce: **03.08.2026**

Platnost zadání bakalářské práce: **30.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Havlina Robin

Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Havlina** Jméno: **Robin** Osobní číslo: **519842**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávající katedra/ústav: **Katedra matematiky**
Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Procedurální generování dungeonů v Godot Engine

Název bakalářské práce anglicky:

Procedural generation of dungeons in Godot Engine

Pokyny pro vypracování:

Cílem práce je navrhnout a implementovat systém procedurálního generování herních dungeonů v enginu Godot s využitím modulárního přístupu. Systém vygeneruje náhodný dungeon z předem vytvořených místností, které jsou propojeny podle pravidel, tak aby zaručila hratelnost a logiku hry. Funkci generátoru potvrdí prototyp jednoduché hry s pixelovým stylem.

Pokyny/Postup:

1. Provedení rešerše existujících metod generování herních úrovní se zaměřením na modulární generování.
2. Analýza typické struktury u dungeonů a návrh pravidel, které definují, jak propojit jednotlivé místnosti a jejich generování.
3. Vytvoření algoritmu generujícího náhodný dungeon z předem připravených místností v enginu Godot.
4. Vytvoření prototypu hry, která tento generátor bude používat.
5. Testování a vyhodnocení funkčnosti generátoru.

Seznam doporučené literatury:

1. Bradfield, C., Godot 4 Game Development Projects. Packt Publishing, 2023.
2. Vanhove, S., Learning GDScript by Developing a Game with Godot 4: A fun introduction to programming in GDScript 2.0 and game development using the Godot Engine. Packt Publishing, 2024.
3. Oficiální dokumentace Godot (Godot Docs). URL: <https://docs.godotengine.org>

DECLARATION

I, the undersigned

Student's surname, given name(s): Havlina Robin
Personal number: 519842
Programme name: Applied Informatics

hereby declare that the Bachelor's Thesis titled

Procedural generation of dungeons in Godot Engine

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 16.07.2026

Robin Havlina

.....
student's signature

Acknowledgment:

I would like to thank for (his/her expert guidance) and express my gratitude to for (his/her language assistance).

Procedurální generování dungeonů v Godot Engine

Studijní program: Aplikovaná informatika

Vedoucí práce: Ing. Petr Pauš, Ph.D., pracoviště školitele (název instituce, fakulty, katedry...)

Klíčová slova: klíčová slova (nebo výrazy) seřazená podle abecedy a oddělená čárkou

Procedural generation of dungeons in Godot Engine

[illegible]

Key words: keywords in alphabetical order separated by commas

Contents

Introduction	8
1 Theoretical Background of Procedural Content Generation	9
1.1 Foundations and Taxonomy of Procedural Content Generation	9
1.1.1 The PCG Taxonomy	9
1.1.2 Dimensions of Control and Parameter Vectors	10
1.1.3 The Challenge of Expressive Range	10
1.1.4 Advantages and Challenges of PCG	10
2 Technology Stack and State of Art	13
2.1 Godot Engine	13
2.1.1 The Node and Scene Composition	13
2.1.2 GDScript	13
2.2 Visuals and Pixel Art Aesthetic	14
2.2.1 The Role of Pixel Art in Roguelike Design	14
2.2.2 Aseprite	14
2.2.3 Workflow and Integration	15
3 Analysis of Algorithms	16
3.1 Random Walk and Agent-Based Generation	16
3.1.1 Theoretical Mechanics and State Space	16
3.1.2 Algorithmic Evaluation	16
3.2 Binary Space Partitioning (BSP) Algorithm	17
3.2.1 Mathematical Space Division and Geometric Partitioning	17
3.2.2 The Connectivity Problem and Delaunay/MST Graph Solutions	17
3.2.3 Algorithmic Evaluation	18
4 Implementation of Generator and UI	19
5 The Game Prototype	20
6 Testing	21
Conclusion	22

Introduction

Procedural Content Generation (PCG) has evolved from techniques used to mitigate hardware limitations in early video games into a widely adopted approach for generating digital content. As players increasingly seek infinite replayability and diverse experiences, the ability to generate game environments dynamically has become an increasingly important tool for game developers, particularly in projects with limited production resources. By leveraging algorithms to create game worlds, developers can overcome the “content bottleneck”—a common design constraint where the scale and variety of a game world are limited by the time and resources required for manual design [1].

This thesis focuses on the research, design, and implementation of a procedural dungeon generation system within the Godot Engine. The primary objective is to compare the practical suitability of bottom-up and top-down procedural generation approaches for roguelike dungeon design, evaluating whether they can provide sufficiently flexible layouts while maintaining stable performance. While many existing solutions focus on a single algorithmic approach, this work explores and contrasts the comparative strengths of two distinct methodologies representing these paradigms: Random Walk and Binary Space Partitioning (BSP).

The core contribution of this work is the development of a modular dungeon generation framework capable of integrating procedural layout generation with gameplay-oriented systems. By utilizing tile bitmasking logic for door connectivity and integrating navigation systems, this thesis demonstrates how procedural layouts can effectively support gameplay mechanics.

Chapter 1

Theoretical Background of Procedural Content Generation

1.1 Foundations and Taxonomy of Procedural Content Generation

Procedural Content Generation (PCG) represents a fundamental framework shift in modern digital game design. As established in modern game design literature [1, 9, 10], PCG is conceptualized as the algorithmic generation of game content with comparatively limited or indirect input from designers [1]. In the context of modern development, this effectively shifts the designer’s role from directly crafting assets to authoring the generative rules and constraints that govern their creation.

1.1.1 The PCG Taxonomy

According to the core literature [1], procedural generators can be classified across several multi-dimensional axes that dictate their operational constraints and architectural purpose:

- **Online vs. Offline Generation:** This axis specifies the temporal execution of the algorithm. Offline generation occurs during the development cycle, serving as an authoring tool to assist designers. In contrast, online generation occurs dynamically at runtime while the game is being executed by the player [1, 9, 10]. The framework implemented in this thesis strictly utilizes *online generation*, forcing the algorithms to operate within strict runtime computation limits to prevent stuttering or long loading screens during level transitions.
- **Necessary vs. Optional Content:** Content is deemed necessary if it is critical to the progression of the game loop, which means that the player must interact with it to advance [1]. Optional content can be bypassed or ignored. Because the generated dungeons dictate the entire layout, progression paths, and enemy configurations necessary to reach the objective, the system developed in this work handles exclusively *the necessary content*. This places an absolute requirement on the algorithmic robustness to prevent dead-ends.
- **Stochastic vs. Deterministic Realization:** A deterministic generator will always produce the exact same output when given identical initial parameters [1]. A stochastic generator relies on probabilistic factors, producing variations in iterations. This project implements a hybrid approach: the internal calculations are stochastic to ensure unpredictable environments, but they are bounded by a fixed random seed, rendering the entire pipeline reproducible.

- **Constructive vs. Generate-and-Test Methods:** Constructive generation runs exactly once and outputs a final level layout without evaluating its aesthetic or structural properties post-generation [1]. Generate-and-test approaches iteratively produce candidates and pass them through a “fitness function” or validator, discarding failures until a viable result is achieved [1]. The algorithms analyzed in this thesis, Random Walk and Binary Space Partitioning (BSP), are primarily *constructive*, though they are frequently augmented with localized topological constraints to guarantee path connectivity [1, 8, 9].

1.1.2 Dimensions of Control and Parameter Vectors

A critical challenge in PCG is the tension between pure algorithmic randomness and structural controllability. Shaker et al. emphasize the necessity of “parameter vectors” or “random seeds,” which function as dimensions of control over the generator’s search space [1].

Without control vectors, a generator risks producing unreadable structural noise. In this work, controllability is achieved by exposing structural parameters to the developer interface. Variables such as total room volume, minimum aspect ratios for spatial partitioning, and branching limits serve as constraints that bound the generative space, allowing the engine to consistently output structurally sound, fair, and engaging layouts.

1.1.3 The Challenge of Expressive Range

Evaluating a content generator requires analyzing its *expressive range*, the structural diversity of the content it is capable of producing [1, 9]. A common pitfall in constructive algorithms is the tendency to generate “variations on a single theme,” where separate iterations look visually and structurally homogeneous. By comparing an agent-based approach (Random Walk) with a geometric decomposition approach (BSP), this thesis explores how different mathematical foundations expand the expressive range of a dungeon generator, offering both organic, chaotic topographies and highly structured, architectural environments within a single system framework.

1.1.4 Advantages and Challenges of PCG

Although the taxonomy defines the mathematical and systemic constraints of a generator, evaluating PCG requires a clear understanding of its practical advantages and inherent software design challenges, as contextualized by the contemporary game design literature [1].

1.1.4.1 Key Advantages of Algorithmic Generation

Implementing procedural generators offers distinct advantages that address both development overhead and player experience.

- **Infinite Replayability:** As established by Shaker et al., replayability is the most prominent driver for online PCG [1]. By producing unique layout topologies for every session, the player cannot rely on pure spatial memory, which shifts the focus of the game toward active adaptation and mechanical mastery.
- **Data Compression and Resource Efficiency:** Historically popularized by titles like *Elite* (1984), PCG allows vast virtual spaces to be represented by minimal storage footprints [1]. Instead of saving thousands of geometric assets or map files, the software only stores the generative algorithm and seed values, reconstructing the data dynamically on demand.

- **Managing Content Diversity and Scalability:** The demands of modern game development often exceed manual production capabilities. Industries utilize hybrid procedural pipelines (e.g. automated foliage generation with tools like SpeedTree) to populate vast environments with diverse variations, avoiding the high financial cost of manually modeling every environmental asset.
- **Parameter-Driven Customization:** Connecting the generator to the control dimensions allows for adjustment of the difficulty in real-time or aesthetic variations [1]. By feeding specific parameters (such as a difficulty vector or visual theme) into the generator, the system can tailor the experience dynamically to match player performance or specific game states.
- **Foundations for Co-Authoring:** Procedural algorithms frequently serve as “co-designers.” A generator can produce a rough structural skeleton (such as a basic dungeon layout or rough terrain), which human level designers subsequently refine, streamlining repetitive tasks and allowing artists to focus on high-value, hand-crafted details.

1.1.4.2 Inherent Challenges and Limitations

Despite its generative power, PCG is not an absolute solution and introduces substantial design risks and structural failings that must be critically managed:

- **Bounded Generative Space vs. Complex Interaction:** As player interaction mechanics become more sophisticated, designing an algorithm capable of accounting for all mechanical variables grows exponentially complex. In many strategy games, players quickly identify the underlying generation rules and default to repetitive, optimized playstyles, reducing the impact of the randomized environment.
- **Incompatibility with Highly Specialized Mechanics:** Games designed around highly precise physics or spatial puzzles (such as *Portal*) rely on tight, intentional environmental layouts. Generating levels that successfully leverage and reward complex player abilities without breaking the game’s logic remains an open research challenge.
- **The Intellectual Vacuum and Lack of Intent:** A fundamental systemic failing of pure mathematical PCG is that algorithms inherently lack artistic design intent. While a human designer places a structural element to convey meaning, trigger an emotional response, or teach a mechanic, a constructive algorithm merely satisfies numerical constraints. Without careful rule-bounding, this vacuum results in spaces that are mathematically valid but fundamentally soulless and design-deficient.
- **The Dynamic Pacing Bottleneck:** Hand-crafted levels allow for precise, authorial control over the emotional and mechanical pacing of the gameplay loop—alternating between high-intensity combat and periods of structural relief. Procedural generation frequently disrupts this curve, occasionally clustering hazardous encounters too closely together or generating vast, unengaging dead zones that break the game’s narrative and mechanical flow.
- **The Aesthetic Homogeneity and Predictability Problem:** Procedurally generated levels can often feel visually flat, repetitive, or structurally chaotic when compared to hand-crafted design. This phenomenon, sometimes referred to as the “thousand oatmeal bowls” problem, highlights how a generator might produce mathematically unique layouts that nonetheless feel indistinguishable and entirely predictable to the player once the underlying logic is decoded. Achieving the visual and atmospheric cohesion of meticulously designed titles (like *Dark Souls*) is incredibly difficult

for pure algorithms, which is why commercial successes like *XCOM 2* or *Invisible, Inc.* rely heavily on modular, rule-bounded constructive generation to maintain a high artistic standard.

Chapter 2

Technology Stack and State of Art

This chapter outlines the technical environment and tools selected to implement and evaluate the procedural generation algorithms. Since the primary focus of this thesis is the algorithmic design of the spatial generators, the selected development stack is configured to minimize rendering and integration overhead, allowing for rapid iteration and performance profiling.

2.1 Godot Engine

The Godot Engine (v4) was selected as the primary platform for executing and rendering the procedural dungeons. Because the engine is lightweight and open-source, it provides a highly flexible environment for implementing custom algorithms without the bloat of larger commercial engines [4].

2.1.1 The Node and Scene Composition

At the core of Godot’s architecture is the “Scene Tree” hierarchy, where game elements are represented as specialized nodes. Unlike rigid component-based systems, Godot favors composition through nested, self-contained scenes. As evaluated by Bradfield [2], this paradigm allows developers to build complex, modular systems by instantiating independent building blocks.

For a procedural dungeon generator, this composition is highly efficient. Instead of forcing the algorithm to handle complex rendering instructions, each room type is authored as an independent, encapsulated scene file (`.tscn`) containing its own static collision bodies, navigation polygons, and visual assets. The generator operates strictly on a logical grid level – calculating room coordinates and topological connections – and then programmatically instantiates these pre-built scenes into the active Scene Tree [2]. This decoupling ensures that changes to a room’s visual design do not require modifications to the underlying generation algorithms.

2.1.2 GDScript

The logical operations driving the generators are written in GDScript, Godot’s proprietary scripting language. In Godot 4, the introduction of GDScript 2.0 brought critical optimizations necessary for data-heavy algorithmic tasks [3].

While procedural generation inherently relies on intensive mathematical operations and iterative array traversals, older dynamic languages often suffer from performance overhead. As analyzed by Vanhove [3], GDScript 2.0 mitigates this through strict static typing, allowing for compiler-optimized

bytecode execution of spatial and multi-dimensional grid coordinates. This provides the runtime speed necessary to generate complex layouts without causing noticeable load-screen delays.

2.2 Visuals and Pixel Art Aesthetic

The visual presentation of a procedurally generated game is a critical factor in how the player perceives the environment. For this project, a pixel-art aesthetic was chosen not only for its distinct stylistic appeal but also for its technical suitability for modular dungeon generation.

2.2.1 The Role of Pixel Art in Roguelike Design

Pixel art is characterized by its reliance on a limited resolution, where each pixel is deliberate and essential to the visual composition. In the context of procedural generation, this aesthetic offers several advantages:

- **High Modularity:** Pixel art is inherently grid-based, making it perfectly compatible with the tile-based nature of dungeon generators.
- **Artistic Consistency:** By strictly adhering to a defined color palette and grid resolution, even randomly assembled dungeon rooms maintain a cohesive and “hand-crafted” appearance.
- **Readability:** In complex, procedurally generated levels, clear visual communication is vital. The simplified shapes of pixel art allow players to quickly distinguish between interactive elements (like doors or levers) and environmental hazards.

2.2.2 Aseprite

Aseprite was selected as the primary tool for creating the visual assets of this prototype. Aseprite is a specialized editor designed specifically for 2D animation and sprite creation, offering features that optimize the workflow for procedural systems as outlined in its comprehensive documentation [6].

The software utilizes a binary file format (.ase) that strictly structures graphic data into layers, frames, and user-defined tags [7]. Understanding this specific internal file structure is highly beneficial when optimizing workflow integration, as it allows for advanced export operations:

- **Sprite Sheets and Metadata:** Aseprite allows for the efficient creation of sprite sheets (atlases) alongside JSON metadata parsed from frame tags [6, 7]. For a procedural generator, this is essential, as the engine can slice a single PNG file into multiple segments, mapping them to different logical IDs based on data extracted from the visual source.
- **Layered Workflow:** The software supports non-destructive layering, which allows the artist to iterate on environmental details (e.g., adding moss to a wall) without altering the base structure.
- **Pixel-perfect Preview:** Aseprite provides immediate feedback on how assets will appear at specific resolutions, ensuring that when the assets are imported into the Godot Engine, they remain crisp and artifact-free.

2.2.3 Workflow and Integration

The workflow from Aseprite to the Godot Engine is streamlined to support the iterative nature of procedural level design. Assets are exported from Aseprite as optimized PNG sprite sheets and imported into Godot's *TileSet* resource editor.

Within Godot, these assets are processed through a "TileSet Atlas" pipeline, which assigns specific physical collision shapes and navigation data to each tile. Because the generation system relies on modular room scenes, this workflow allows for a rapid update cycle: an artist can modify a sprite in Aseprite, re-export the file, and the Godot generator automatically updates the look of the dungeon without requiring changes to the underlying algorithm or code structure. This efficiency is paramount when fine-tuning the visual variety of the dungeon's environments.

Chapter 3

Analysis of Algorithms

3.1 Random Walk and Agent-Based Generation

The Random Walk algorithm, often formalized in game design as an “agent-based digger” or the “Drunkard’s Walk,” is a bottom-up stochastic process. The algorithm can be modeled as a discrete-time Markov process [12].

3.1.1 Theoretical Mechanics and State Space

In a standard two-dimensional grid environment, the algorithm initializes a generative agent at a starting position $P_0 = (x_0, y_0)$. At each discrete execution step t , the agent transitions to a neighboring state P_{t+1} by selecting a random cardinal direction from the set of available vectors:

$$D = \{(0, 1), (0, -1), (1, 0), (-1, 0)\}$$

The position is updated dynamically via the stochastic function, where d is sampled uniformly from D :

$$P_{t+1} = P_t + d, \quad d \in D$$

As the agent occupies a coordinate, it mutates the cell state from a structural wall to an open, navigable floor tile. The process continues until a termination threshold defined either by a maximum step counter l_{max} or a specific targeted room volume is reached [12].

3.1.2 Algorithmic Evaluation

- **Advantages:** The agent-based approach inherently guarantees local connectivity along the generated path. Because every floor tile is carved sequentially by a generative agent, the resulting layout is free of completely detached, unreachable segments. Under the specific experimental constraints defined by Naußed and Sapokaite, the Drunkard’s Walk algorithm yielded a playable relative space size of 100%, as the sequential carving process prevented the generation of isolated walkable tiles in their testing environment [12]. Additionally, the algorithm is computationally inexpensive; from an asymptotic perspective, its time complexity scales linearly as $O(l_{max})$ where l_{max} is the maximum step threshold, with each iteration requiring $O(1)$ state checks and coordinate mutations under the assumption of constant-time grid access. The worst-case space complexity is similarly bounded by $O(l_{max})$ to store the coordinates of the visited grid cells.

- **Limitations:** While local paths are connected, the global shape is highly unpredictable and difficult to constrain. Without bounding envelopes, the layout can easily clip through pre-defined world boundaries. Furthermore, establishing a structured progression path or logically distributing critical gameplay checkpoints requires separate graph-traversal post-processing, as the basic algorithm does not account for macro-level spatial flow.

3.2 Binary Space Partitioning (BSP) Algorithm

Binary Space Partitioning (BSP) is a top-down geometric subdivision method, often employing randomized split locations in PCG applications to achieve structured, architectural layouts [13].

3.2.1 Mathematical Space Division and Geometric Partitioning

The core mechanic of the BSP algorithm is the recursive subdivision of a continuous or discrete two-dimensional bounding region V_0 into disjoint rectangular sub-regions along randomly selected horizontal or vertical axes [13].

During each recursive pass, the algorithm evaluates the dimensions of a generated sub-region. To keep the generative process stable, the subdivision is governed by defined geometric limits: a maximum division depth, a minimum permissible sub-region width (W_{min}), and a minimum permissible sub-region length (H_{min}) [13]. If a region exceeds these thresholds, a cutting coordinate k is selected within bounded constraints to prevent heavily skewed, narrow slices [13]. The space is then subdivided into two smaller disjoint sub-regions, V_A and V_B , such that:

$$V_A \cap V_B = \emptyset, \quad V_A \cup V_B = V_{parent}$$

The partitioning terminates automatically when no further subdivisions are possible, either because the maximum iteration limit is reached or because the dimensions of all remaining sub-regions fall below the permitted minimum bounds [13]. The finalized sub-regions serve as the spatial boundaries for room placement. However, because these partitioned regions are adjacent and tightly packed, specific geometric constraints, namely room offsets and corner modifiers, are applied to define a structural buffer zone between neighboring walls to prevent room overlaps and reserve space for subsequent corridor integration [13].

3.2.2 The Connectivity Problem and Delaunay/MST Graph Solutions

Because top-down partitioning inherently segments space into geometrically separated room regions, a primary challenge of BSP is corridor generation and connectivity assurance [13]. Naive implementations that attempt to connect adjacent partitions locally often suffer from a lack of reliable routing, which may fail to preserve global connectivity [13].

To resolve this connectivity problem, some modern BSP-based implementations augment the partitioning process with graph-theoretical approaches to establish a reliable topological layout. For analytical purposes, let $G = (V, E)$ denote the complete graph where $V = \{v_1, v_2, \dots, v_n\}$ represents the spatial centers of all valid generated rooms $v_i = (x_i, y_i) \in \mathbb{R}^2$, and E represents the set of all possible straight-line paths connecting these vertices. For each edge $e = (v_i, v_j) \in E$, a scalar weight $w(e)$ is assigned, representing the Euclidean distance between the room centers:

$$w(e) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

From this representation, the topological connectivity is established through a multi-stage graph-theoretic pipeline:

1. **Delaunay Triangulation:** A triangulation $DT(V) \subseteq G$ is computed such that no vertex $v_k \in V$ lies inside the circumcircle of any triangle in $DT(V)$. This step generates a planar graph limiting candidate connections to spatially adjacent rooms, reducing the likelihood of geometrically implausible connections.
2. **Minimum Spanning Tree (MST):** To avoid a chaotic, over-connected maze of corridors, Kruskal's or Prim's algorithm is applied to the triangulation graph $DT(V)$ to extract a Minimum Spanning Tree $T = (V, E_{MST}) \subseteq DT(V)$. The tree T is defined as the acyclic subgraph that spans all vertices while minimizing the total sum of edge weights while preserving connectivity:

$$\text{Cost}(T) = \sum_{e \in E_{MST}} w(e)$$

This guarantees graph connectivity between all room vertices.

3. **Loop Re-insertion:** A pure MST creates a strictly acyclic layout, meaning there is only one valid path between any two points. In game design, this may increase backtracking and reduce navigational flexibility. To optimize exploration and tactical routing, a small subset of discarded non-MST Delaunay edges $E_{cycle} \subset (DT(V) \setminus E_{MST})$ is often deliberately re-inserted into the final graph, depending on the desired topological density of the layout:

$$G_{final} = (V, E_{MST} \cup E_{cycle})$$

3.2.3 Algorithmic Evaluation

- **Advantages:** BSP provides structured spatial distribution and predictable control over environment scale and density [13]. It is well suited for generating structured, architectural layouts and naturally lends itself to room-based progression metrics.
- **Limitations:** The rigid, perpendicular alignment of rooms and corridors can appear blocky or visually repetitive if not masked by diverse art assets. Structurally, BSP does not inherently guarantee connectivity between generated rooms, requiring specialized path-validation frameworks or secondary graph-theoretic routing to resolve [13]. Furthermore, the secondary graph compilation layer increases computational complexity compared to the lightweight Random Walk. From an asymptotic perspective, the geometric partitioning phase operates proportional to the size of the partition tree, which contains $O(2^d)$ nodes in the worst case, where d represents the maximum division depth. This results in both time and space complexity bounded by $O(2^d)$ for the partitioning step. Generating candidate connections via Delaunay triangulation for $n = |V|$ room vertices requires $O(n \log n)$ time. Because the resulting triangulation $DT(V)$ is a planar graph, its edge count is bounded by $|E_{DT}| \leq 3n - 6$, enabling the subsequent Minimum Spanning Tree construction to be completed within $O(n \log n)$ time complexity. Consequently, BSP-based generation trades increased computational complexity for greater control over spatial organization.

Chapter 4

Implementation of Generator and UI

Chapter 5

The Game Prototype

Chapter 6

Testing

Conclusion

Bibliography

- [1] N. Shaker, J. Togelius, M. J. Nelson: *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016.
- [2] C. Bradfield: *Godot 4 Game Development Projects*. Packt Publishing, 2023.
- [3] S. Vanhove: *Learning GDScript by Developing a Game with Godot 4: A fun introduction to programming in GDScript 2.0 and game development using the Godot Engine*. Packt Publishing, 2024.
- [4] Godot Engine Documentation: *Godot Docs*. [Online]. Available: <https://docs.godotengine.org/> [Accessed: July 2026].
- [5] Godot Engine Documentation: *GDScript Basics*. [Online]. Available: https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html [Accessed: July 2026].
- [6] Igará Studio: *Aseprite Docs*. [Online]. Available: <https://www.aseprite.org/docs/> [Accessed: July 2026].
- [7] Igará Studio: *Aseprite File Format Specifications*. [Online]. Available: <https://github.com/aseprite/aseprite/blob/main/docs/ase-file-specs.md> [Accessed: July 2026].
- [8] M. H. M. Al-Khafaji, et al.: *Procedural 2D Dungeon Generation Using Binary Space Partition Algorithm And L-Systems*. *Journal of Computer Science*, 2024.
- [9] K. Stuart, et al.: *Graph-Based Procedural Dungeon Generation for Roguelike Games*. *International Journal of Computer Games Technology*, 2024.
- [10] S. R. Research, et al.: *Visual Computing and Optimization Frameworks for Procedural Level Generation*. *International Journal of Computing and Multimedia*, Vol. 6(2), September 2025.
- [11] T. Short: *3 Failings of Procedurally Generated Game Design*. Game Developer, 2014. [Online]. Available: <https://www.gamedeveloper.com/design/3-failings-of-procedurally-generated-game-design> [Accessed: July 2026].
- [12] D. Naußed, R. Sapokaite: *Evaluation of Procedural Content Generators for Two-Dimensional Top-Down Dungeon Levels*. Bachelor's thesis, Department of Game Design, Uppsala University, 2021. [Online]. Available: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1569492> [Accessed: July 2026].
- [13] J. M. Yap, N. C. Woo, B. S. Tan, K. S. Chong: *From Algorithm to Playable Space: A Technical Note on BSP-Based Dungeon Design*. *International Journal of Creative Multimedia*, Vol. 6(2), pp. 287–308, 2025.